DISTRIBUTED MULTI-REASONING MECHANISM TO SUPPORT CONCEPTUAL STRUCTURAL DESIGN

By Lucio Soibelman[1] and Feniosky Peña-Mora,[2] Members, ASCE

ABSTRACT: The conceptual phase of structural design involves selecting preliminary materials, selecting the overall structural form of the building, producing a rough dimensional layout, and considering technological possibilities. Decisions are made on the basis of such information as height of the building, building use, typical live load, wind velocity, earthquake loading, design fundamental period, design acceleration, maximum lateral deflection, spans, story height, and other client requirements. More detailed information about the task itself, constraints, possible solution principles, and known solutions for similar problems is extremely useful in the process of defining and finding a solution to the design problem. This paper presents the M-RAM, which is intended to assist engineers in the conceptual phase of the structural design of tall buildings by providing designers with adapted past design solutions generated by a distributed multi-reasoning mechanism. The objective of the M-RAM (Multi-Reasoning Model) is to provide the adapted past design solutions in an organized and reliable way, creating a support system to enhance creativity, engineering knowledge, and experience of designers. To test the feasibility of the proposed model, a prototype of a distributed artificial intelligence system was developed in which the Internet was used as a communication backbone for the different systems that implemented the reasoning mechanisms.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

INTRODUCTION

The writers of this paper take the view that design is an information-processing activity. After each information output, it might become necessary to improve on the result of the previous step and to repeat each step until the necessary improvements have been achieved. Every design first involves a full understanding of the problem and what information is known about it. The knowledge, ability, and experience of the designers are obviously important factors in developing the initial design, but for any designer, detailed information about the specific task, constraints, possible solution principles, and known solutions for similar problems is necessary to identify the precise nature of the design requirements.

The conceptual design phase is the first step in the design process. After knowing the problem definition, a designer develops an initial design estimate consistent with the designer's experience and knowledge, the constraints, and the requirements. The efficiency and success of the design process depends heavily on this initial estimate. Because this design phase requires knowledge, use of past experience, rules of thumb, intuition, and so forth, it is very difficult to use any procedural programming language for its computerization (Maher 1987).

In the conceptual design phase, past experience plays a vital role, where the process is often assisted by identifying, retrieving, and modifying appropriate past design cases and generalizing the cases, with the implied benefit of being able to use relevant past experience (Manfaat et al. 1997).

This paper presents both the M-RAM (Multi-Reasoning Model) (Soibelman 1998) and a prototype that was developed to evaluate the feasibility of the proposed model. The model and prototype are intended to assist engineers in the initial stages of the structural design of tall buildings by providing designers with adapted past design solutions generated by a distributed multi-reasoning mechanism. The final objective is to help designers to employ past experience and avoid past mistakes. On the basis of functional, architectural, and aesthetic requirements, M-RAM arrives at a few possible configurations for the structural systems, mainly based on geometry and load distribution for gravity and lateral loads, such as dead, live, wind, and earthquake loading.

The conceptual design is a complex problem that cannot be solved by only one reasoning mechanism. When facing this challenge, designers use a combination of heuristic cases, first principles, and past experience to reduce the complexity of the problem and to come up with possible solutions. Instead of building general function modules, the M-RAM model applies an agentlike approach to develop competence modules, each of which is an expert at a particular small task. Each module does all representation, computation, reasoning, and execution related to its particular competence. The emphasis in the proposed architecture is on a more direct coupling of perception to action, distribution, and decentralization, dynamic interaction with the environment, and intrinsic mechanisms to cope with limitations in resources and incomplete knowledge. The traditional positive aspects of distributed processing systems, such as performance, reliability, modularity, and resource-sharing, motivate the development of this architecture (Maes 1991). The application of this approach allows us to overcome the difficulties presented by traditional procedural programming languages to automate or support the conceptual phase of structural design for tall buildings.

Although many different machine-learning algorithms and reasoning mechanisms have been applied before to support the development of conceptual designs, the user had to mediate those programs by hand (Maher and Fenves 1985; Maher 1987; Fenves et al. 1995; Maher and de Silva Garza 1996; Manfaat et al. 1997). With M-RAM, research focused on examining the interconnection and distribution of existing algorithms, mechanisms, and models incorporating existing application programs through a computer network, taking advantage of the Internet's large potential.

CONCEPTUAL PHASE OF STRUCTURAL DESIGN PROCESS

Although people have been engaged for centuries in engineering design, only in the past few decades has this activity been perceived as a systematic process capable of comprehensive analysis and improvement. The penetration of knowledge-

[1]Asst. Prof., Dept. of Civ. and Envir. Engrg., Univ. of Illinois at Urbana-Champaign, Urbana, IL 62822.

[2]Assoc. Prof., Dept. of Civ. and Envir. Engrg., Massachusetts Inst. of Technol., Cambridge, MA 02139.

Note. Associate Editor: James H. Garrett Jr. Discussion open until November 1, 2000. To extend the closing date one month, a written request must be filed with the ASCE Manager of Journals. The manuscript for this paper was submitted for review and possible publication on February 17, 1999. This paper is part of the ***Journal of Structural Engineering***, Vol. 126, No. 6, June, 2000. ©ASCE, ISSN 0733-9445/00/0006-0733–0742/$8.00 + $.50 per page. Paper No. 20260.

based tools into the design process is widely perceived as leading to fundamental changes, not only in the accuracy and speed in which a design is achieved, but also in the design process itself.

The engineering design process is primarily a synthesis analysis repetition. The engineer synthesizes (formulates) the problem and then analyzes it. If certain constraints are not met, then the engineer goes back to the synthesis part, changes the formulation, and then analyzes again, and so on, until the specific constraints are met (Pouangare 1986). This synthesis analysis loop is also denoted as the design-analysis cycle.

The conceptual phase of structural design is a typical synthesis problem, and its most important output is the definition of the structural system. The structural system can be defined as an arrangement of structural components and subsystems to properly transmit the forces from the superstructure to the substructure subject to constraints on geometry, flexibility, ductility, durability, and cost. The structural system must have the ability to transmit the forces both globally and locally. In the conceptualization phase, the structural designer is concerned only with the global transmission of forces.

In the early 1970s, computers began to be applied as a helpful tool for structural design, but not until the early 1980s were tools developed to support the synthesis part of the design problem. This led to the situation where the engineer did the synthesis part and the computer did the analysis part of the design (Pouangare 1986).

In the mid-1980s, with the growth of the science of artificial intelligence, researchers initiated attempts to automate the synthesis part of the design problem. Different expert systems were developed. One of the most innovative, called HI-RISE, was developed by Maher and Fenves from Carnegie-Mellon University (Maher and Fenves 1985). Like most expert systems developed at that time, HI-RISE fell short in the learning process. Knowledge acquisition and lack of flexibility were the main problems faced by expert systems such as HI-RISE, and few of these models have been implemented as commercial tools.

In the 1990s, with the development of new artificial intelligence tools, research efforts were initiated to automate the synthesis part of the conceptual phase of structural design. Work done by Fenves, Rivard, Gomez, and Chiou (Fenves et al. 1995) applied case-based reasoning to allow designers to save solutions as a byproduct of the design process. Such cases could then be retrieved in later ''similar'' design situations and adapted to the new situation. Bailey and Smith (1994) used case-based design in the CADRE system as an approach for integrating preliminary architectural and structural design. CADRE retrieves a case in which design integration has already been achieved, and the design process is therefore one of adapting the case to achieve new goals. Maher and de Silva Garza (1996) went one step further, applying genetic algorithms to adapt cases retrieved by the case-based reasoning tool. During this decade, several distributed applications, environments, and testbeds were developed. Lander and Lesser (1992) developed TEAM, a multiple-agent system that supports the integration of heterogeneous and reusable agents; Talukdar, DeSouza, Quadrel, and Ramesh (1992) developed A-TEAM, a multiple-agent organization for cyclic (iterative) processing that has been applied to solve high-rise building design; Killicote and Garrett (1998) developed ASPEN, an agent-based framework for providing computer-aided support for using design standards; and Gasser and Hill (1990) developed MACE, a language, programming environment, and testbed for distributed artificial intelligence systems.

The M-RAM research has built upon the work done by those researchers. The main difference resides in the approach. While the main goal of those researchers was to support agent collaboration, aiding the different participants in the design process, the goal for M-RAM was to study the feasibility of applying agents to support the subprocesses of a divided structural design process. It is recognized that complex problems can be solved by breaking them into smaller problems. Dividing a problem into its smallest unit parts has the objective of imitating the way that the human brain deals with complex problems. According to Minsky (1988), each mind is made of many smaller processes, called agents. Each mental agent by itself can only do some simple thing that needs no thought process at all. Yet when those agents are joined in societies—in certain very special ways—this leads to true intelligence (Minsky 1988). Like the human brain when dealing with complex problems, M-RAM uses different reasoning mechanisms to solve each part of the divided design problem. First, a model was developed to allow a better understanding of the problem requirements. This led to a distributed agent architecture implementation providing a ''plug and play'' approach with easy system maintenance and evolution, as will be demonstrated in the next two sections of this paper.

M-RAM

The M-RAM (Fig. 1) is represented using Unified Modeling Language (UML) (Rational Software Corporation 1997) object-oriented modeling. The model describes the structure of the objects in the system and their identity, relationships to other objects, attributes, and operations. It demonstrates the way in which the multi-reasoning distributed artificial intelligent system selects the characteristics of artifacts such as the structural material, form, system, and parameters for tall building design. The model does so by first choosing the most suitable structural system using a classification agent (C4.5), then supplying different possible structural forms and coefficients from cases provided by the case-based reasoning (CBR) agent, and finally by adapting the retrieved cases by applying a genetic algorithm (GA) agent.

Designer

A designer represents the entity (human, e.g., a structural engineer; or a computer, e.g., a structural engineer assistant) that presents a proposal based on a design intent and/or an artifact characteristic that needs to be satisfied. An example would be a designer who, after receiving information on and specifications from the architect, for the building to be built, defines structural coefficients such as loads (e.g., live, wind, and earthquake), design fundamental period, and maximum lateral deflection based on past experience, local standards, and some structural formulae. All this information is used as input for the next structural design stage: the dimensioning of the structural components.

Proposal

A proposal represents the statement given by the computer designer. A proposal includes two important elements: (1) The recommendation believed to satisfy a design intent and required artifact characteristics; and (2) the justifications for why the recommendation fulfills that design intent and/or those artifact characteristics.

Intent

Design intent refers to what the designer needs to achieve or satisfy (Peña-Mora et al. 1995). Design intent has a ranking that specifies its importance and a satisfaction measure that specifies the extent to which the design intent has been satisfied by a recommendation. Design intent refers to the follow-

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

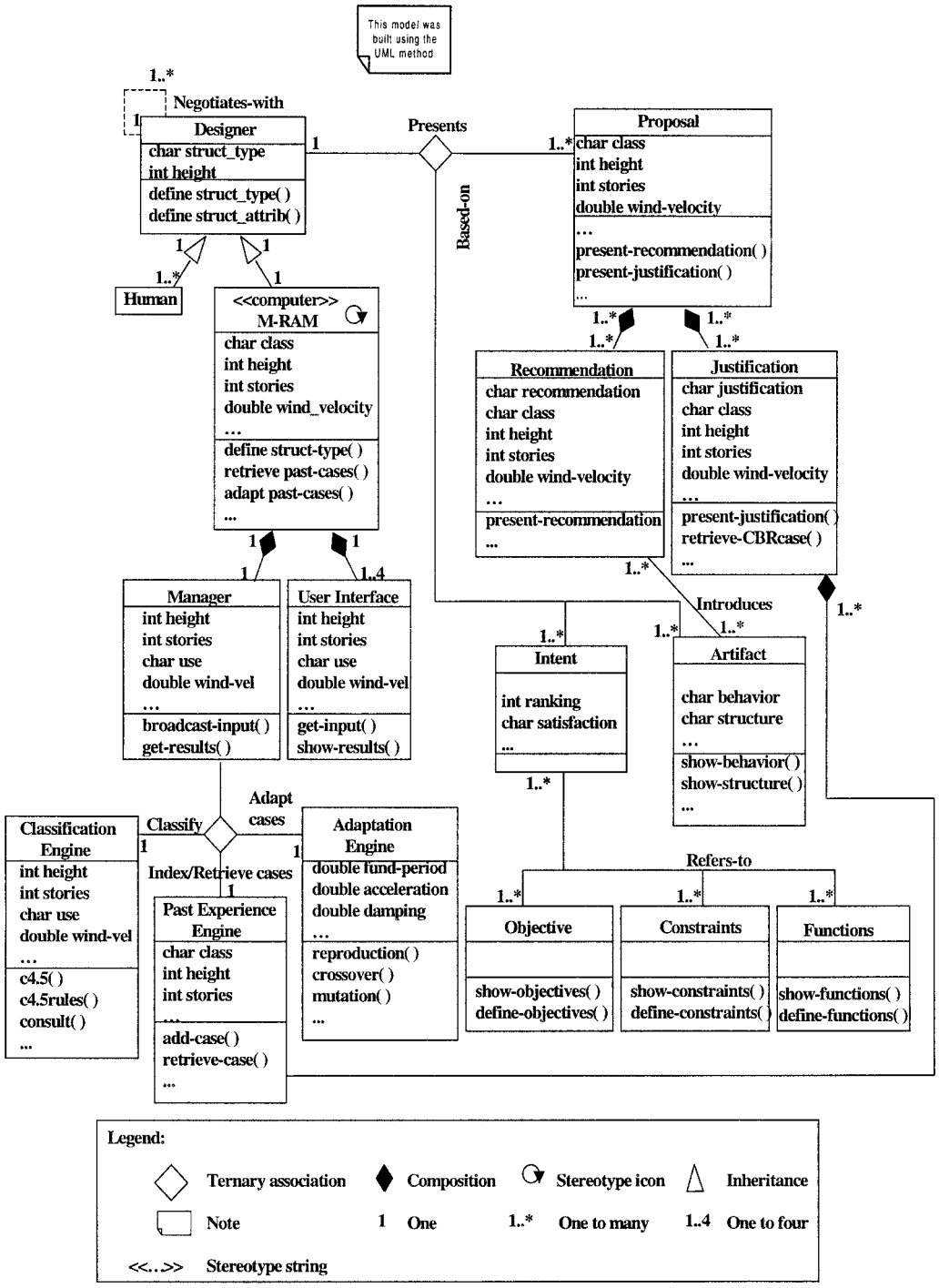


FIG. 1. M-RAM Object Model. Graphical Notation of UML Model (Rational Software Corp. 1997)

ing attributes of a design: (1) objectives; (2) constraints; and (3) functions.

- ''Objective'' is a characteristic to be optimized by the artifact. It presents a measure against which the design is checked (e.g., minimize the construction cost). Designers tend to use this class of intents as an evaluation, which requires comparison among several competing designs.
- ''Constraints'' is a confinement of an artifact (e.g., lack of skilled labor in certain regions, making it very difficult to build concrete structures). In this class of intents, designers only need to test if the criterion is met by the design recommendation without the need to compare it to other design alternatives.
- ''Function'' is an activity performed by an artifact (e.g., to safely withstand the earthquake loads during the building lifetime). This class establishes the performance criteria that later translates to the behavior of the system and specific constraints.

Recommendation

Recommendation refers to the entity that satisfies the design intents and/or required artifact characteristics. A recommendation is a proposed structural system presented by the classification agent, the cases presented by the CBR agent, and the adaptation of the retrieved cases presented by the GA agent. Several feasible structural systems could be proposed for the same tall building.

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

Justification

Justification refers to the reason that partially explains why a recommendation will satisfy a design intent. According to Peña-Mora (1994), a justification could be a rule (e.g., if the number of floors <40, then a braced-frame structural system is feasible); a case (e.g., this building is similar to the Sears Tower in Chicago); a catalogue (e.g., this structural system solution was taken from an entry in the *Structural Systems for Tall Buildings* published by the Council on Tall Buildings and Urban Habitat (Kowalczyk et al. 1995); a principle (e.g., Hooke's law); an authority (e.g., this structural system has been suggested by someone who is an authority in the field); a trade-off (e.g., this is the best possible design based on the trade-off between minimizing the cost and minimizing the deflection); a prototype (e.g., the model of the proposed structure produced these measurements in the wind tunnel); a constraint network (e.g., this condition satisfies all the constraints on the system); or a Pareto optimal surface (e.g., this design falls on the surface of the best possible design when optimizing cost, schedule, and ecological impact). A justification may support other justifications by providing supporting evidence or assumptions. Cases of past experience provided by the CBR engine are used as justification in the M-RAM implementation.

Computer—M-RAM

The M-RAM computer implementation is a distributed artificial intelligence computer system developed as a decision support tool for the preliminary design of tall building structures. Decisions are made on the basis of such information as height of the building, building use, region, typical live load, wind velocity, earthquake loading, design fundamental period, design acceleration, maximum lateral deflection, spans, story height, and other client requirements. It runs on the Internet using a World Wide Web (WWW) interface.

Manager and User Interface

The Internet is being used as a communication backbone for the different systems that implement the reasoning mechanisms being employed. Each reasoning mechanism considered as an autonomous module can be installed in different servers. Each distributed server acts as an intelligent agent. A WWW interface receives the expected data as input and launches a manager program, which broadcasts the problem to all existing modules. Each module solves a part of the problem and sends back its partial solution to the manager program. This programs sends the solution or possible solutions back to the WWW interface, which displays it to the user. In the current implementation, M-RAM was developed with three modules: the classification module implemented by the C4.5 agent; the past cases/experience module implemented by the CBR agent; and the adaptation module implemented by the GA agent.

Classification Engine

In the M-RAM implementation, a C4.5 agent (Quinlan 1993) implemented the classification engine. In C4.5, recorded classifications are examined and a model is constructed inductively by generalization from specific examples. This machine-learning algorithm can be defined as a set of computer programs that construct classification models by discovering and analyzing patterns found in given records. The algorithm that has fundamental importance in C4.5 is the one that generates the initial decision tree from a set of training cases. Since the cases do not all belong to the same class, a divide-and-conquer algorithm attempts to split them into subsets using statistical evaluation tests and an information gain ratio criterion to find the best possible tree structure (Quinlan 1993).

The C4.5 agent can be divided into a set of training cases, a decision tree, and a set of decision rules generated by simplification of the decision tree. The decision tree can be subdivided into leaves, which indicate classes, and decision nodes, which specify some test to be carried out on a single attribute value (Fig. 2).

For the practicing structural engineer, the cataloguing of structural systems for tall buildings has historically recognized the primary importance of the system to resist lateral loads (Kowalczyk et al. 1995). At one end of the spectrum are the moment-resisting frames, which are efficient for buildings in the range of 20 to 30 stories; at the other end is the generation of a tubular system with high cantilever efficiency that is good for 70 to 80 stories. Any particular form is economical for a limited range of building heights.

Recognizing this historical knowledge, M-RAM applies C4.5 to create a decision tree and a set of rules with the objective of clustering the database and consequently reducing the search space. The database, obtained from the monographs edited by the Council on Tall Buildings and Urban Habitat (Kowalczyk et al. 1995), was classified into five classes according to the system adopted to resist lateral loads: braced frame and moment-resisting frame, shear wall, core and outrigger, tubular, and hybrid. This class division was defined by the available cases. The M-RAM classification agent is flexible enough to allow the creation of any other category, according to future available cases.

In the M-RAM C4.5 implementation, each case concerns one design. Nineteen properties have been used to describe each case: (1) height from street to roof; (2) number of stories; (3) number of levels below ground; (4) building use; (5) frame material; (6) typical floor life load; (7) basic wind velocity; (8) maximum lateral deflection; (9) design fundamental period—transverse; (10) design fundamental period—longitudinal; (11) design acceleration; (12) design damping; (13) earthquake loading; (14) typical floor story height; (15) typical floor beam span; (16) typical floor beam depth; (17) typical floor beam spacing; (18) column size at ground floor; and (19) column spacing. The outcome of each case is given as braced frame and moment-resisting frame, shear wall, core and outrigger, tubular, or hybrid, depending on which kind of structural system was applied to resist lateral loads.

From the available cases, C4.5, acting as a learning system, constructed a classification model that relates the type of structural system to resist lateral loads to the values of the recorded properties. There are 46 cases of actual built tall buildings in the dataset. In the implementation, the same database is used to build the decision tree model and to create the past experience case database. We recognize the fact that these data are not sufficient to induce a good generalization. A robust pattern cannot be distinguished from chance coincidences, but the objective in this stage of the research was to define the viability of the proposed system architecture. The next step in this research is to augment the dataset; consulting engineers and computer simulation can be used to accomplish this.

Past Experience Engine

A case-based reasoning agent implemented the past experience engine. Case-based reasoning is the process of ''remember and compare'' (Kolodner 1993). A reasoner remembers previous situations similar to the current one and uses them to help solve a new problem. The knowledge of a case-based reasoner is constantly changing as new experience gives rise to new cases that are stored for future use. A case-based reasoner learns from experience to exploit prior successes and avoid repeating past failures (Leake 1996).

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

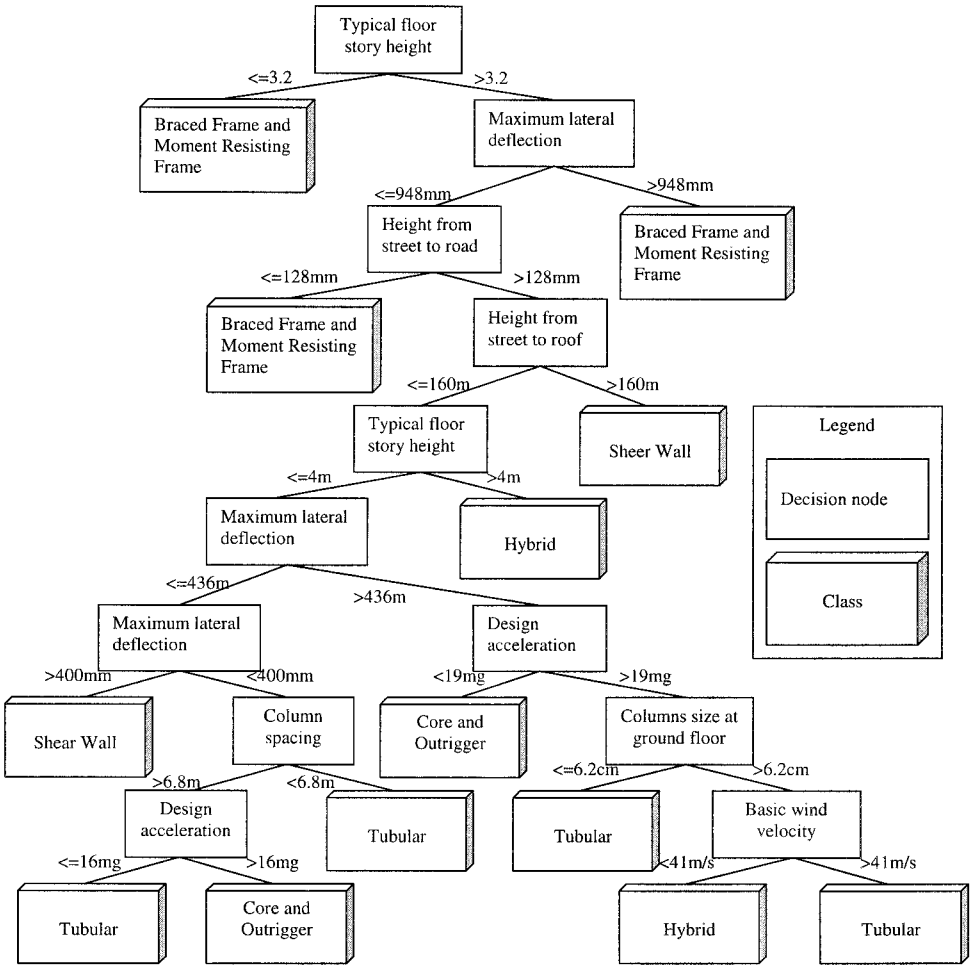


FIG. 2. M-RAM C4.5 Tree

The CBR agent can be divided into three parts: a case library, an indexing engine, and a retrieving engine. The case library contains the case problem description (e.g., the state of the world at the moment the case was happening and, if appropriate, what problem needed solving at the time), the case solution description (e.g., the stated or derived solution to the problem specified in the problem description, or the reaction to the situation), and the case outcome (e.g., the resulting state of the world after the solution was carried out). The indexing engine can be subdivided into two parts: the case-labeling engine and the library organization engine. The case-labeling engine ensures that cases can be retrieved at appropriate times. In general, labels designate under what circumstances the case might have a lesson to teach and therefore when it is likely to be useful. The library organization engine organizes the cases so that the search through the case library can be done efficiently and accurately. Finally, the retrieving engine can be subdivided into the searching engine (e.g., finding potentially matching cases) and a matching engine (e.g., each case is judged for its potential usefulness).

In the M-RAM implementation, the manager, after receiving the classification from the C4.5 agent, requests the CBR agent to retrieve the most useful case. The CBR agent returns to the manager the home page of the retrieved case. From this page, the designer can follow the links to complete information about the retrieved case.

Caspian (''Introductory'' 1996b), a case-based reasoning system written in C and developed at the University of Wales —Aberystwith, was chosen to be implemented as the M-RAM CBR agent. Caspian creates a case base from a case file written in the language CASL (''Creating'' 1996a). The contents of a case base are described in a file known as a case file. The program Caspian uses this case file to create a case base in the computer memory, which can then be accessed and modified to retrieve the most suitable design solution to the problem being solved using case-based reasoning techniques. When new cases are added to the case base, they are appended to the end of this case file.

Caspian requires the cases to be indexed according to one or more attributes. The program searches the case base for the subset of cases that matches all the index constraints exactly. In the M-RAM implementation, the cases were indexed according to their structural system type. The program then searches the case base for all cases that match the class previously defined by the M-RAM classification agent.

After the index search has been completely successfully, Caspian can be run in two different modes: with or without case matcher. If the case matcher is invoked, the program scans the subset of cases to find the one with the highest weight value. The weight is calculated using the same 19 properties that were used to describe each case in the classification agent. If the case matcher is not invoked, the program retrieves the whole subset of cases that matches the indexed constraints. In the M-RAM implementation, Caspian is run in both modes. First, the program invokes the case matcher to retrieve the case that is the best fit to the design problem being analyzed; then the program is run again without the case matcher, with the objective of retrieving all cases that match the class defined by the M-RAM's classification agent. All retrieved cases are then used as the initial generation in the M-RAM adaptation agent.

J. Struct. Eng. 2000.126:733-742.

Gene	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Value	211.9	47	2	5	1	5	30	1000	4.52	4.73	52	1.5	0.06	3.58	17.5	850	6.4	4500	6.4

Where:

0 Height from the street to the roof
1 Number of stories
2 Number of levels below ground
3 Building use: 1=office; 2=hotel; 3=residential; 4=garage; 5=mixed
4 Frame material: 1=steel; 2=concrete; 3=mixed
5 Typical floor life load
6 Basic wind velocity
7 Maximum lateral deflection
8 Fundamental-period transverse
9 Fundamental-period longitudinal
10 Design acceleration
11 Design damping
12 Earthquake loading
13 Typical floor story height
14 Typical floor beam span
15 Typical floor beam depth
16 Typical floor beam spacing
17 Columns size at ground floor
18 Columns spacing

FIG. 3. Building Genome Representation

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

Adaptation Engine

Genetic algorithms (GA) provide a robust yet efficient search methodology, explicitly modeled upon the biological ''survival of the fittest'' reproductive model. In the broadest sense, a GA creates a set of solutions that reproduce based on their fitness in a given environment. The process follows the following pattern: (1) An initial population of random solutions is created; (2) each member of the population is assigned a fitness value based on its evaluation against the current problem; (3) solutions with a higher fitness value are most likely to parent new solutions during reproduction; and (4) the new solution set replaces the old, a generation is complete, and the process continues from step 2. That sequence implements, in a most simplistic way, the concept of the survival of the fittest. The outcome of a genetic algorithm is based on probabilities, just as biological success is grounded in chance. The standard model for a GA solution is a bit string called a chromosome, after its biological counterpart. During reproduction, the chromosomes of parent solutions combine and undergo mutation in creating the next generation.

The GA agent can be subdivided into a phenotype (e.g., a design solution) and a genotype (e.g., a way of representing or encoding the information that is used to produce the phenotype) (Koza 1996). The principle of the survival of the fittest determines whether a genotype survives to reproduce. The first step in preparing to solve a GA problem is the identification of a suitable representation scheme. After the phenotype is converted into a genotype, the GA solves the problem with the help of three basic operations: reproduction, crossover, and mutation.

In the M-RAM implementation, the partially matching retrieved design cases provided by the CBR agent are used as the initial population (i.e., generation 0) for the GA. The retrieved design cases that are represented using attribute-value formalism describe a specific design solution and correspond to the building's genotype. Here it is important to consider that, unlike the traditional implementation of GA, where the initial (e.g., seed) population has a few extraordinary individuals within a population of mediocre colleagues, the M-RAM deals with the above-average initial population provided by the CBR agent. In this GA implementation, what is needed is just a fine tuning. This implies that the M-RAM adaptation engine relies heavily on reproduction and crossover, having a very low mutation rate.

GAlib, a C++ genetic algorithm library developed at MIT, was used to implement M-RAM's GA agent (Wall 1996). The library includes tools for using genetic algorithms to do optimization in any C++ program using any representation and any genetic operators. The GAlib library is centered around two main classes: a genome and a genetic algorithm. Each genome represents a single solution to the problem. The genetic algorithm object defines how the evolution should take place. The genetic algorithm uses an objective function to determine how ''fit'' each genome is for survival. It uses genome operators (e.g., built into the genome) and selection/replacement strategies (e.g., built into the genetic algorithm) to generate new individuals. The genetic algorithm object determines which individuals should survive, which should reproduce, and which should die. It also records statistics and decides how long the evolution should continue.

The objective of the M-RAM's GA agent is to combine bits and pieces from several past experiences retrieved from the CBR agent in order to solve a new problem. The attribute-value representation of cases retrieved by M-RAM's CBR agent is reinterpreted and used in the context of the genetic algorithm by considering the attributes used in the description of a case to be equivalent to genes. Thus, the collection of attributes used to describe a specific building corresponds to the building's genome. The values that are associated with each of the 19 attributes in the description of a case make up the case description and can be defined as the case phenotype (Fig. 3).

The type of genetic algorithm applied by M-RAM's GA agent is the steady-state algorithm implemented by GAlib's GASteadyStateGA class. This genetic algorithm uses overlapping populations with a user-specified amount of overlap. The algorithm creates a population of individuals by cloning the population passed by M-RAM's CBR agent. In each generation the algorithm creates a temporary population of individuals, adds these to the previous population, then removes the worst individuals in order to return the population to its original size. Unlike most genetic algorithm implementations that create a random initial population, this approach allowed the cloning of the cases provided by the CBR agent as an initial population. The amount of overlap could be specified to allow the required fine tuning by maintaining most desired characteristics of the above average cases provided by the CBR agent.

The objective function is an adaptation of the one presented by Maher and de Silva Garza (1996). Two types of considerations are used to measure a solution's fitness in M-RAM's GA agent: the problem requirements, and the general structural design constraints (represented by attribute correlation). Both types of fitness are represented as constraints, and the objective is measured by the number of constraints not violated. The solution is feasible if none of the constraints are violated; therefore, the goal of the GA agent is to maximize constraint nonviolations.

Two examples of constraint are

1. Problem requirement constraint: If 300 m is the required height of the building being designed, the height is then considered to be one of the design constraints. To evaluate a genome proposed by the genetic algorithm, the objective function adds 10 points to the objective value of all genomes that have height in the interval (285–315 m), 5 points to genomes that have height in the intervals (240–285 m) and (315–360 m), and nothing to all other genomes.
2. General structural design constraint: To retrieve feasible

J. Struct. Eng. 2000.126:733-742.

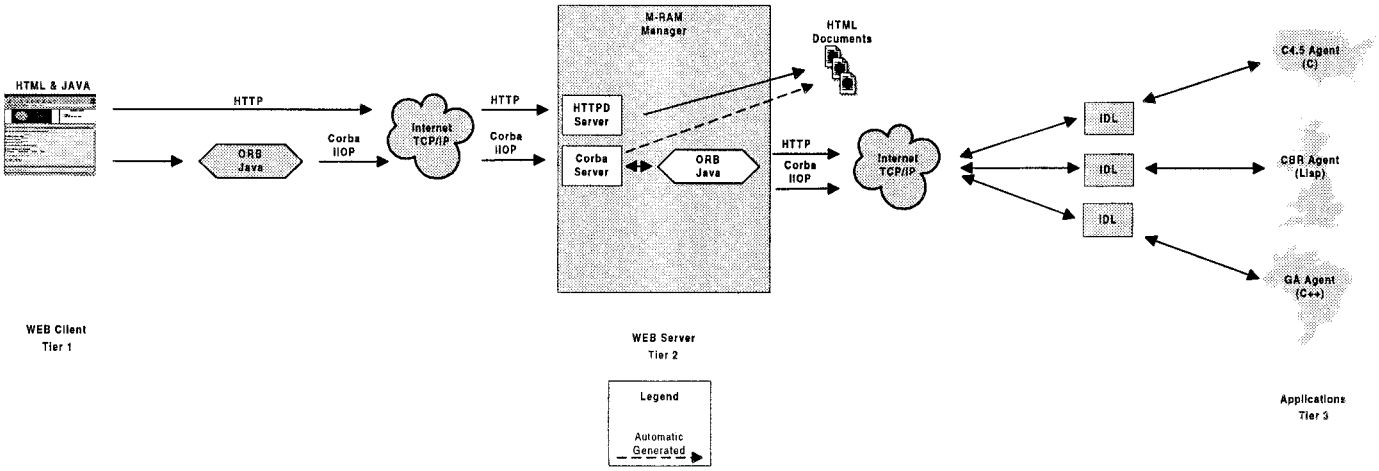


FIG. 4. M-RAM Architecture

design solutions, the objective function gives points to solutions that keep the proportionality among number of floors, story height, and building height. The objective function gives 10 points to all genomes that respect the following equation:

$$0.95 * \text{Story Height} < (\text{Height of the Building}/\text{Number of Floors}) < 1.15 * \text{Story Height}$$

M-RAM ARCHITECTURE AND IMPLEMENTATION

Getting information processes to share a common syntax is a major problem. Fig. 4 demonstrates how M-RAM deals with this problem, presenting the architecture that was applied in the prototype development. One of the most important characteristics of the proposed architecture is the possibility of geographical dispersion of the reasoning mechanisms. The idea was to allow the individual agents to be developed and maintained in different parts of the world using a ''plug-and-play'' architecture, leaving the agents where they were developed. This allows a faster client response that does not need to download all components and databases to its machine and allows easy agent maintenance and upgrade.

In the actual M-RAM implementation, a WWW interface written in Java (e.g., first tier) receives the expected data as input and launches the manager program (e.g., second tier). The manager program decides which server will perform which operation on the data, sending the required data to the location where the application is situated (e.g., third tier). Each application solves a part of the problem, sending back this partial solution to the manager program. The manager's task is to coordinate those partial solutions, providing a consistent solution for the problem as a whole. Finally, the manager program sends the solution or possible solutions of the structural design problem back to a WWW interface, which displays it to the user.

In this implementation, Java provides the mobile code foundation, and the Common Object Broker Architecture (CORBA) (Mowbray and Ruh 1997) provides the distributed object infrastructure. CORBA objects can be defined as binary components that remote clients can access via method invocations. Clients don't need to know where the distributed object resides, what operation a system executes on, or how the server object is implemented. What the client needs to know is the interface the server object publishes. This interface serves as a binding contract between client and servers (Orfali and Harkey 1997). CORBA-distributed object technology allows us to put together complex client/server information systems by simply assembling and extending components.

In the M-RAM architecture, CORBA's Object Request Broker (ORB) is the object bus. It lets objects transparently make requests to and receive responses from other objects, locally or remotely. It contains real-time information describing the function a server provides and its parameters. The ORB intercepts the call and is responsible for finding an object that can implement the request, pass it the parameters, invoke its methods, and return the results. The client just needs to know the object interface.

M-RAM enforces collaboration by writing code for the two sides of the collaboration. The next step would be the development of intelligent components that do more than just interoperate, but instead collaborate at the semantic level to get a job done. The trick, however, is to get components with no previous knowledge of each other to collaborate. To get to that point, we will need standards that set the rules of engagement for different component interaction boundaries. The Knowledge Query and Manipulation Language (KQML) (Finin et al. 1992), which is both a message format and a message-handling protocol to support run-time knowledge-sharing among agents, can provide these standards for a future implementation of an advanced M-RAM model.

ILLUSTRATIVE EXAMPLE

To test the feasibility of the proposed model, an M-RAM prototype was developed. To see how the prototype works and its design recommendations, consider, for example, the design of a hypothetical office building in Boston, Massachusetts. The specifications of the new design are for 85 floors, but because the building will be located in Boston's financial area near Logan Airport, the architect wants to keep the total height of the building as low as possible. To do so, he defined the total height to be around 280 m. The specifications also require that the structural material be steel because of New England's cold winters and labor characteristics, making the execution of massive concrete structures difficult and expensive. Finally, because of space requirements, the architect is also specifying 12 m of space between columns.

To be able to run M-RAM with these information requirements, the designer first needs to open the M-RAM project home page at http://cee-ta.mit.edu. This home page includes an open M-RAM interface button, clicking on which causes the M-RAM interface (Fig. 5) to pop onto the computer screen.

This easy-to-use interface allows the designer to interact with the three available agents. If the designer has any doubt about the input data, the available interface help can be used. Clicking the interface help menu automatically opens a browser with information about M-RAM attributes.

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

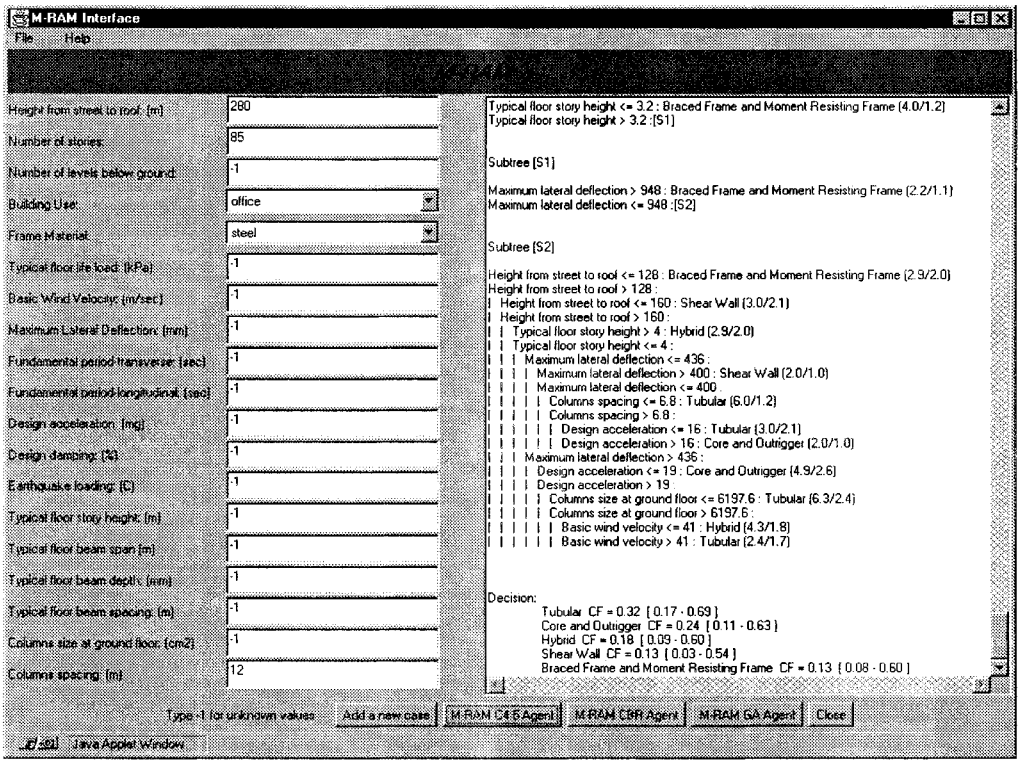


FIG. 5. M-RAM Interface with C4.5 Agent Output

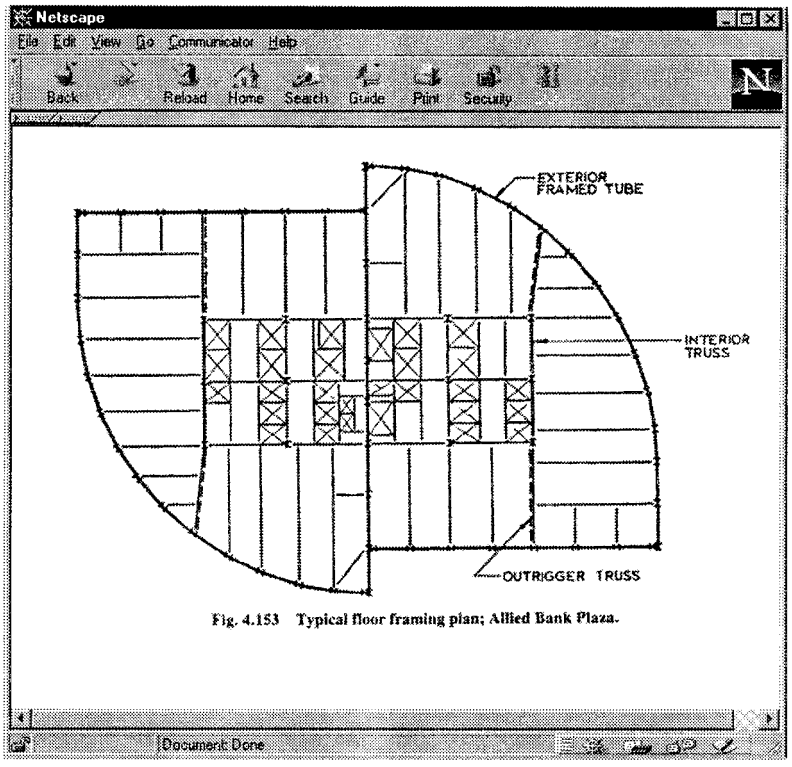


FIG. 6. Structural Detail of Example's Matching Case

To run the classification agent, the designer needs to click in the M-RAM C4.5 agent button after typing into the M-RAM interface the values for the design problem requirements (typing "−1" for unknown values). When the designer clicks the M-RAM C4.5 agent button, the values of the attributes are sent to the manager, which locates the classification agent and sends the information to the server where the C4.5 engine is located. The C4.5 engine classifies the design problem, sending the classification solution back to M-RAM manager, which displays it in the text area of the M-RAM interface, as shown in Fig. 5.

The output of the classification agent can be divided into

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

three parts. The first part prints the information input by the designer, the second presents the actual decision tree, and the third presents the classification results. The example design (e.g., height from the street to the roof = 280 m; number of stories = 85; building use = office; frame material = steel; column spacing = 12 m; and all other attributes = −1) was classified as a tubular system with a probability somewhere between 0.17 and 0.69, the best guess being 0.32 [certainty factor (CF) is equal to 0.32]. CF is obtained during the testing stage of C4.5's training process, where the lower and upper bond and the confidence interval are defined for each class. M-RAM is thus informing the designer that the majority of buildings with characteristics similar to the one being designed were designed with tubular structural systems to resist lateral loads. With this information, M-RAM is ready to run its CBR agent to retrieve from its database of building designs the one that is the best fit, compared with the example design. To do so, the designer just needs to press the M-RAM CBR agent button. The manager then sends the design attributes added to the class defined by the C4.5 agent to the server where the CBR agent is located. The CBR agent retrieves from its memory the URL of the best matching case and sends this information back to the manager, which displays it to the user in the M-RAM interface text area. The CBR agent retrieved as a matching case for the design example a case instance called Allied Bank Plaza in Houston, Texas, linking the design solution to the retrieved case WWW URL. With this URL, the M-RAM manager launches a browser window that pops up in the designer computer with the home page of the retrieved case. From this home page, the user can go to a text description of the retrieved case, to a page with structural information on the case in tabular format, to a page with graphical structural details (Fig. 6), and to a page with an index of all cases of the same class as the one being designed. From this index, the designer is able to browse among available cases.

The retrieved case is a building with height from the street to the roof equal to 296 m, 71 floors, 4 levels below ground, office use, structural steel, 2.5 kPa of typical floor life load, 196 kN/m of basic wind velocity, H/500 maximum lateral deflection for 100 years return, 1% serviceability for design damping, story height equal to 4 m, 15.2 m of beam span, 0.53 m of beam depth, 4.6 m of beam spacing, column size of 1,016 by 610 mm in perimeter and 610 by 610 mm interior, and columns spacing of 4.6 m perimeter and 9.15 by 6.1 m interior. By comparing the problem requirements with the retrieved case attributes, it is possible to see that the retrieved case matches the specifications on the following attributes: height from the street to the roof, building use, and structural material. The retrieved case is a building approximately as tall as the one being designed, built with the same structural material and for the same use. Unfortunately, the retrieved case doesn't completely match the specifications for the other two requirements: number of floors and column spacing.

The building being designed has almost the same total height compared with the retrieved case, but because it has more floors, it is possible to conclude that it is going to have shorter floor-to-floor heights. The retrieved solution does not help the designer to deal with shorter floor-to-floor heights and larger distance among columns. The third M-RAM agent is then used to transform the retrieved solution by mixing parts of partial matches available in M-RAM's database, retrieved by the CBR agent.

To do so, the designer needs to press the M-RAM GA agent button. The manager then sends the design attributes that will be used as input to the objective function. The genetic algorithm agent runs for 1,500 generations, mixing bits and parts of the design cases provided by the CBR agent, and sends its solution back to the manager, which displays it to the user in the M-RAM interface text area.

The output of the genetic algorithm agent can be divided into two parts: The first prints statistical information about the genetic algorithm, and the second presents the genetic algorithm-proposed solution to the design problem. The M-RAM recommendation is a building with height from the street to the roof equal to 275 m, 82 floors, 4 levels below ground, office use, structural steel, 3.0 kPa of typical floor life load, 52 m/s of basic wind velocity, 522 mm maximum lateral deflection, fundamental period—transverse of 5 s, fundamental period—longitudinal of 6.5 s, design acceleration of 27 mg, 2% serviceability for design damping, story height equal to 3 m, 23 m of beam span, 1.63 m of beam depth, 4.9 m of beam spacing, column size of 70,728 cm^2, and column spacing of 12.15 m. It proposed a tall building with the same use, structural material, and nearly the same height, number of floors, and column spacing as that indicated in the problem specification. As expected, it has shorter floor-to-floor height, compared with the case retrieved by the CBR agent.

M-RAM classifies a design based on incomplete structural information. It retrieves a case from past experience, providing some insight into what occurred in the past so that previous mistakes can be avoided. Finally, it provides some recommendations for the values of the unknown structural parameters. M-RAM generates possible design alternatives, based on the specifications and available knowledge. This allows designers to evaluate different alternatives.

CONCLUSIONS

The use of the model and prototype presented in this paper improves the design process in time and quality. The required design-analysis cycle does not need to be repeated several times during the design process because M-RAM provides the designer with a better initial input for the design process. In addition, the ability of the system to retrieve lessons learned in previous, similar designs allows the designer to correct problems, avoiding repeating past mistakes in new designs. Usually the information at the conceptual design stage is not sufficient for an engineering analysis to determine the feasibility of a proposed solution. The M-RAM model supports designers by providing needed information in an organized and reliable structure.

This model is an improvement over the existing design process. Designers depend solely on their knowledge and experience to provide the assumptions they make during the initial stages of structural design, as well as the specifications for the design artifact. These initial assumptions have a strong impact on the quality of the final design and consequently on the quality of the final artifact. This model improves the conceptual design by supporting the designer's knowledge and experience. The well-organized database added by different functions provided by the model helps the designer to classify the design problem, retrieve useful past experience, and adapt past experience, providing the needed information to improve the quality and speed of the design process. In addition, the model provides a structural representation for providing information to the designer. Hence, the users will not be exposed to radical changes in their work technologies or be replaced by an automated computer designing system. This model supports the actual design process by providing the required information that helps the designer to elaborate on the design.

M-RAM can help not only the professional designers, but also students. It could be a very powerful tool to support structural engineering education, since M-RAM would be able to help students to quickly assimilate knowledge by learning with past design examples. Special tutors can easily be developed as an add-on to the proposed model.

In summarizing this work, the main contributions can be divided into two parts. First, a model for representing the con-

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

ceptual phase of structural design was developed. This model provides initial building blocks for representing design knowledge in terms of the reasoning process used by designers in generating an artifact satisfying their design intents. This model also takes into consideration the different reasoning mechanisms that collaborate to solve the design problem and is used to provide active computer support for designers in the reasoning process.

The second contribution was the development of a distributed open-system architecture that integrates different reasoning mechanisms. This novel system architecture was applied where different reasoning mechanisms can be located in different servers dispersed in diverse locations around the world. This research had the objective of integrating agents that were not intended to collaborate to reach a common solution. It provided a solution that achieves a given goal with the integration of different reasoning mechanisms, providing a solution to a problem by using the power of cooperation. It also provided a framework in which collaboration could occur while reducing overhead of maintenance, limitation of scalability, or danger of obsolescence in such a rapidly changing field. And finally, it uses a combination of graphs, text, and tables to convey the information, with the easy possibility of adding simulation capabilities without a major change in the proposed system architecture.

APPENDIX. REFERENCES

Bailey, S. F., and Smith, I. F. C. (1994). ''Case-based preliminary building design.'' *J. Computing in Civ. Engrg.*, ASCE, 8(4), 454–468.

''Creating a case-base using CASL.'' (1996a). University of Wales—Aberystwith, U.K.

Fenves, S. J., Rivard, H., Gomez, N., and Chiou, S. C. (1995). ''Conceptual structural design in seed.'' *J. Arch. Engrg.*, ASCE, 1(4), 179–186.

Finin, T., Fritzson, R., and McKay, D. (1992). ''A language and protocol to support intelligent agent interoperability.'' *Proc., CE&CALS Conf.*, Society for Computer-Aided Engineering, Rockford, Ill.

Finin, T., Fritzson, R., McKay, D., and McEntire, R. (1994). ''KQML as an agent communication language.'' *Proc., 3rd Int. Conf. on Information and Knowledge Mgmt.*, ACM Press, New York.

Gasser, L., and Hill, R. W. (1990). ''Coordinated problem solver.'' *Annual review of computer science*, Vol. 4, J. F. Traub, B. J. Grosz, B. W. Lampson, and N. J. Nilsson, Annual Reviews Inc., Palo Alto, Calif., 203–253.

An introductory guide to Caspian. (1996b). University of Wales—Aberystwith, U.K.

Kiliccote, H., and Garrett Jr., J. H. (1998). ''Agent-based support for building code usage in building design systems.'' *Proc., Struct. Engrs. World Congr.*, Elsevier Science, New York.

Kolodner, J. (1993). *Case-based reasoning*, Morgan Kaufmann, San Mateo, Calif.

Kowalczyk, R. M., Sinn, R., and Kilmister, M. B. (1995). *Structural systems for tall buildings*, Council on Tall Buildings and Urban Habitat —McGraw-Hill, New York.

Koza, J. R. (1996). *Genetic programming—On the programming of computers by means of natural selection*, MIT Press, Cambridge, Mass.

Lander, S. E., and Lesser, V. R. (1992). ''Customizing distributed search among agents with heterogeneous knowledge.'' *Proc., 1st Int. Conf. on Information and Knowledge Mgmt.*, Y. Yesha, ed., Calgary, Canada. 335–344.

Leake, D. B. (ed.). (1996). ''CBR in context: The present and the future.'' *Case-based reasoning—Experience, lessons, and future directions*, AAAI Press/MIT Press, Cambridge, Mass.

Maes, P. (1991). *Designing autonomous agents: Theory and practice from biology to engineering and back*, MIT Press, Cambridge, Mass.

Maher, M. L. (1987). ''Expert systems for structural design,'' *J. Computing in Civ. Engrg.*, ASCE, 1(4), 270–283.

Maher, M. L., and de Silva Garza, A. G. (1996). ''The adaptation of structural systems designs using genetic algorithms.'' *Information processing in civil and structural engineering design*, B. Kumar, ed., Civil-Comp Press, Edinburgh, U.K.

Maher, M. L., and Fenves, S. J. (1985). ''HI-RISE: An expert system for the preliminary structure design of high rise buildings.'' *Knowledge engineering in computer-aided design*, J. S. Gero, ed., North-Holland, Amsterdam.

Manfaat, D., Duffy, A. H. B., and Lee, B. S. (1997). *SPIDA for the utilization of layout design experience*, University of Strathclyde, Scotland.

Minsky, M. (1988). *The society of mind*, Touchstone/Simon and Schuster, New York.

Mowbray, T. J., and Ruh, W. A. (1997). *Inside Corba—Distributed object standards and applications*, Addison-Wesley Longman, Inc., Reading, Mass.

Orfali, R., and Harkey, D. (1997). *Client/server programming with Java and Corba*, Wiley, New York.

Peña-Mora, F. (1994). ''Design rationale for computer supported conflict mitigation during the design-construction process of large-scale civil engineering systems.'' PhD thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Peña-Mora, F., Sriram, D., and Logcher, R. (1995). ''Design rationale for computer-supported conflict mitigation.'' *J. Comp. in Civ. Engrg.*, ASCE, 9(1), 57–72.

Pouangare, C. C. (1986). ''Strategies for the conceptual design of structures.'' MS thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Quinlan, J. R. (1993). *C.4.5 programs for machine learning*, Morgan Kaufmann, San Mateo, Calif.

Rational Software Corporation. (1997). ''Unified modeling language—Notation guide, UML summary, process-specific extensions, and UML semantics.'' ⟨*http://www.rational.com*⟩.

Soibelman, L. (1998). ''The exploration of an integrated representation for the conceptual design phase of structural design for tall buildings through distributed multi-reasoning algorithms.'' PhD thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Talukdar, P. S., DeSouza, R., Quadrel, R., and Ramesh, V. C. (1992). ''A-teams: Multi-agent organizations for distributed iteration.'' *Tech. Rep. EDRC-18-30-92*, Engrg. Des. Res. Ctr., Carnegie Mellon University, Pittsburgh, Pa.

Wall, M. (1996). *GAlib: A C++ library of genetic algorithm components, Version 2.4, Revision B.* Mech. Engr. Dept., Massachusetts Institute of Technology, Cambridge, Mass.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

J. Struct. Eng. 2000.126:733-742.